Unit III -8051 Programming in C

3.1 Embedded C Programming:

Why C:

Compilers produce hex files that are downloaded into ROM of microcontroller. The assembly language produces a hex file that is much smaller than C. Assembly language programming is tedious and time consuming. C is less time consuming and much easier to write.

Microcontrollers have limited on-chip ROM
C is easier to modify and update
You can use code available in function libraries

Advantages of C programming:

a. Easier and less time consuming to write in C than assembly programming.
b. C is easier to modify and update
c. You can use code available in function libraries
d. C code is portable to other microcontroller with little or no modification

Data Types of 8051 C:

A good understanding of C data types for 8051 can help programmers to create smaller hex files.

Data Types in Embedded C Data Types in C Programming Language help us declaring variables in the program.

The data types in Embedded C area

a) unsigned char
b) signed char
c) signed int
d) unsigned int
e) Sbit(Single bit)
f) Bit
g) Sfr :

Unsigned Char: Unsigned char is 8 bit data type in the range 0 – 255 (0 - FF).
This is widely used because 8051 is a 8 bit microcontroller. This is used as ASCII characters and to count the value.

Signed char: Signed char is an 8 bit data type in which the MSB represents the sign (+ or -) in the range - 128 to +127.

Unsigned int: Unsigned int is a 16 bit data type which takes the value in the range 0 to 65535 (0 – FFFF). This is used for – □ Defining 16 bit memory addresses □ To set the counter values which is more than 256

Signed int: Signed int is a 16 bit data type. MSB is the sign bit and the remaining 15 bits represent the magnitude from - 32768 to +32767.

sbit: This data type is used in case of accessing a single bit addressable registers. It allows to access to the single bit SFR registers .

Bit : This data type is used for accessing the bit addressable memory of RAM (20h-2fh).

Sfr: This data type is used for accessing a SFR register by another name. All the SFR registers must be declared with capital letters

Various data types of C:

i. Unsigned char
ii. Signed char
iii. Unsigned int
iv. Signed int
v. Sbit (single bit)
vi. Bit and sfr

i. Unsigned char:
Since 8051 is an 8-bit microcontroller, the character data type is the most natural choice for many applications and most widely used data types for 8051. It is an 8-bit data type that takes a value in the range of 0 – 255(00H – FFH). It occupies one byte location in internal RAM. In setting a counter value, to store string of ASCII characters, where there is no need for signed data, unsigned char should be used instead of signed char. Default is signed char. „unsigned" keyword is used to declare unsigned char.

Write an 8051 C program to send values 00 – FF to port P1.
Solution:

```
#include <reg51.h>
void main(void)
{
unsigned char z;
for (z=0;z<=255;z++)
P1=z;
}
```

Write an 8051 C program to send hex values for ASCII characters of 0, 1, 2, 3, 4, 5, A, B, C, and D to port P1.
Solution:

```
#include <reg51.h>
void main(void)
{
unsigned char mynum[]="012345ABCD";
unsigned char z;
for (z=0;z<=10;z++)
P1=mynum[z];
}
```

ii. Signed char:
It is an 8-bit data type that uses MSB to represent the – (negative) or + (positive) values. Lower 7 bits are used to represent magnitude of the signed number from -128 to +127. It is used to represent quantity like temperature.

Write an 8051 C program to send values of –4 to +4 to port P1.
Solution:

```
#include <reg51.h>
void main(void)
{
char mynum[]={+1,-1,+2,-2,+3,-3,+4,-4};
signed char z; for
(z=0;z<=8;z++)
P1=mynum[z];
}
```

iii. Unsigned int:
It is a 16-bit data type that takes value in the range of 0 to 65535 (0000H – FFFFH). It is used to define 16-bit variables like memory address and to set counter values of more than 256. It occupies two bytes of location in internal RAM. If one byte location is sufficient, char data type should be used instead of int because the size of hex file will be smaller with the use of char data type. „unsigned" keyword is used to declare unsigned int otherwise signed int is the default type of int.

Write an 8051 C program to toggle bit D0 of the port P1 (P1.0) 50,000 times.
Solution:

```
#include <reg51.h>
sbit MYBIT=P1^0;
void main(void)
{
unsigned int z;
for (z=0;z<=50000;z++)
{
MYBIT=0;
MYBIT=1;
}
}
```

iv. Signed int:
It is a 16-bit data type that uses the MSB to represent – (negative) or + (positive) values. Lower 15 bits are used to represent the magnitude of the number from -32768 to +32767.

v. Sbit (Single bit):
„sbit" keyword is used to access single-bit addressable Special Function Registers including P0 to P3.

Write an 8051 C program to toggle only bit P2.4 continuously without disturbing the rest of the bits of P2.
Solution:

```
#include <reg51.h>
sbit mybit=P2^4;
void main(void)
{
while (1)
{
mybit=1; //turn on P2.4
mybit=0; //turn off P2.4
}
```

```
}
```

vi. Bit and sfr:
„bit‟ data type allows access to single bits of bit-addressable memory of internal RAM from 20H to 2FH. „sfr‟ data type is used to access byte-size SFR registers.

Write an 8051 C program to get the status of bit P1.0, save it, and send it to P2.7 continuously.
Solution:

```
#include <reg51.h>
sbit inbit=P1^0;
sbit outbit=P2^7;
bit membit; //use bit to declare bit- addressable memory
void main(void)
{
while (1)
{
membit=inbit; //get a bit from P1.0
outbit=membit; //send it to P2.7
}
}
```

Data Type	Size in Bits	Data Range/Usage
unsigned char	8-bit	0 to 255
(signed) char	8-bit	-128 to +127
unsigned int	16-bit	0 to 65535
(signed) int	16-bit	-32768 to +32767
sbit	1-bit	SFR bit-addressable only
bit	1-bit	RAM bit-addressable only
sfr	8-bit	RAM addresses 80 – FFH only

Time Delay:
There are two ways to create delay:
a. Using simple for loop
b. Using timers

Factors which affect delay using for loop:
1. 8051 uses 12 clock periods per machine cycle. Newer generation of 8051 like DS5000 uses 4 clock periods per machine cycle, DS89C420 uses only one clock periods per machine cycle. Lesser the number of clock periods per machine cycle, faster the operation of microcontroller, lesser the delay.

2. The crystal frequency connected to XTAL1 and XTAL2 input pins. The duration of clock period for machine cycle is a function of crystal frequency.

3. C compiler converts C program to assembly language instructions. Different compilers produce different code and hence produce different hex code.
Write an 8051 C program to toggle bits of P1 continuously forever with some delay.
Solution:

```
#include <reg51.h>
void main(void)
{
unsigned int x;
for (;;)                    //repeat forever
{
p1=0x55;
for (x=0;x<40000;x++);          //delay size //unknown
p1=0xAA;
for (x=0;x<40000;x++);
}
}
```

Write an 8051 C program to toggle bits of P1 ports continuously with 250 ms.
Solution:

```
#include <reg51.h>
void MSDelay(unsigned int);
void main(void)
{
while (1) //repeat forever
{
P1=0x55;
MSDelay(250);
p1=0xAA;
MSDelay(250);
}
}
void MSDelay(unsigned int itime)
{
unsigned int i,j;
for (i=0;i<itime;i++)
for (j=0;j<1275;j++);
}
```

Logic operations in 8051 C:
One of the most important and powerful features of the C language is its ability to perform bit manipulation.

Byte-wise operators in C:
a. AND (&&),
b. OR (||),
c. NOT (!)

Bit-wise operators in C:
a. AND (&),
b. OR (|),
c. EX-OR (^),
d. Inverter (~),
e. Shift Right (>>),
f. Shift Left (<<)

Bit-wise shift operation in C:
a. Shift Right (>>),
b. Shift Left (<<)

Format: data >> number of bits to be shifted right
data << number of bits to be shifted left.

Example: 0x9A >> 3 = 0x13.

These operators are widely used in software engineering for embedded systems and control.

Run the following program on your simulator and examine the results.
Solution:

```
#include <reg51.h>
void main(void)
{
P0=0x35 & 0x0F; //ANDing
P1=0x04 | 0x68; //ORing
P2=0x54 ^ 0x78; //XORing
P0=~0x55; //inversing
P1=0x9A >> 3; //shifting right 3
P2=0x77 >> 4; //shifting right 4
P0=0x6 << 4; //shifting left 4
}
```

Write an 8051 C program to toggle all the bits of P0 and P2 continuously with a 250 ms delay. Using the inverting and Ex-OR operators, respectively.
Solution:

```
#include <reg51.h>
void MSDelay(unsigned int);
void main(void)
{
P0=0x55;
P2=0x55;
while (1)
{
P0=~P0;
P2=P2^0xFF;
MSDelay(250);
}
}
```

Write an 8051 C program to convert packed BCD 0x29 to ASCII and display the bytes on P1 and P2.
Solution:

```
#include <reg51.h>
void main(void)
{
unsigned char x,y,z;
unsigned char mybyte=0x29;
x=mybyte&0x0F;
//Mask lower 4 bits
P1=x|0x30;    //Make it
ASCII
y=mybyte&0xF0; Mask
upper 4 bits

y=y>>4;   //Shift it
to lower 4 bits
```

```
P2=y|0x30;
//Make it ASCII
}
```

Write an 8051 C program to convert ASCII digits of '4' and '7' to packed BCD and display them on P1.
Solution:

```
#include <reg51.h>
void main(void)
{
unsigned char bcdbyte;
unsigned char w=„4";
unsigned char z=„7";
w=w&0x0F;
/ / M a s k   3
w=w<<4; //shift left to
make upper BCD digit
z=z&0x0F;
/ / M a s k   3
bcdbyte=w|z;
P1=bcdbyte;
//combine to make
packed BCD
}
```

In 8051 data can be stored in three types of memory:

1. The 128 bytes of RAM space with address range 00H to 7FH.

2. The 64 kbytes of on-chip code or program memory with address range 0000H to FFFFH. Here both program as well data can be stored and accessed using program counter (PC). To read data from this memory "movc a, @a+dptr" instruction is used in assembly language program. To store data into this memory DB (define byte) assembler directive is used. If more data is stored in this memory, less space is left for program. This space is used to store predefined data and tables. But data cannot be written to it during execution of program.

3. The 64 kbytes of external memory can be used as both RAM and ROM. MOVX instruction is used to access external memory.

<u>RAM data space usage by 8051 C compiler:</u>

C compiler allocates RAM locations in the following order:
1. Bank 0 – address 00H to 07H
2. Individual variables – addresses 08H and beyond
3. Array elements – addresses right after variables
4. Stack – addresses right after array elements.

Write, compile and single-step the following program on your 8051simulator. Examine the contents of the code space to locate the values.
Solution:

```
#include <reg51.h>
void main(void)
{
unsigned char mydata[100]; //RAM space
unsigned char x,z=0;
for (x=0;x<100;x++)
{
z--;
mydata[x]=z;
P1=z;
}
}
```

Compile and single-step the following program on your 8051 simulator. Examine the contents of the 128-byte RAM space to locate the ASCII values.
Solution:

```
#include <reg51.h>
void main(void)
{
unsigned char mynum[]="ABCDEF"; //RAM space
unsigned char z;
for (z=0;z<=6;z++)
P1=mynum[z];
}
```

<u>Accessing Code ROM Space in 8051 C:</u>

To store data in code space or memory instead of RAM memory, keyword *code* is used in front of the variable declaration

```
Ex:     code unsigned char num[] = "1234";
        code unsigned char weekdays=7, month=0x12;
```

Compile and single-step the following program on your 8051 simulator. Examine the contents of the code space to locate the ASCII values.
Solution:

```
#include <reg51.h>
void main(void)
{
code unsigned char mynum[]="ABCDEF";
unsigned char z;
for (z=0;z<=6;z++)
P1=mynum[z];
}
```

Compare and contrast the following programs and discuss the advantages and disadvantages of each one.
(a)

```
#include <reg51.h>
void main(void)
```

```
{ P1=„H
“;
P1=„E“;
P1=„L“;
P1=„L“;
P1=„O“;
}
```

Short and simple, but the individual characters are embedded into the program and it mixes the code and data together.

(b)
```
#include <reg51.h>
void main(void)
{
unsigned char mydata[]="HELLO";
unsigned char z;
for (z=0;z<=5;z++)
P1=mydata[z];
}
```
Use the RAM data space to store array elements, therefore the size of the array is limited.

(c)
```
#include <reg51.h>
void main(void)
{
code unsigned char mydata[]="HELLO";
unsigned char z;
for (z=0;z<=5;z++)
P1=mydata[z];
}
```
Use a separate area of the code space for data. This allows the size of the array to be as long as you want if you have the on-chip ROM. However, the more code space you use for data, the less space is left for your program code.

Sample C Programs:

Write an 8051 C program to toggle all the bits of P1 continuously.
Solution:

```
#include <reg51.h>
void main(void)
{
for (;;)
{
P1=0x55;
P1=0xAA;
}
```

LEDs are connected to bits P1 and P2. Write an 8051 C program that shows the count from 0 to FFH (0000 0000 to 1111 1111 in binary) on the LEDs.
Solution:

```
#include <reg51.h>
#defind LED P2;
void main(void)
{
```

```
P1=00; //clear P1
LED=0; //clear P2
for (;;) //repeat forever
{
P1++; //increment P1
LED++; //increment P2
}
}
```

Write an 8051 C program to get a byte of data form P0. If it is less than 100, send it to P1; otherwise, send it to P2.
Solution:

```
#include <reg51.h>
void main(void)
{
unsigned char mybyte;
P0=0xFF; //make P0 input port
while (1)
{
mybyte=P0; //get a byte from P0
if (mybyte<100)
P1=mybyte; //send it to P1
else
P2=mybyte; //send it to P2
}
}
```

Write an 8051 C program to monitor bit P1.5. If it is high, send 55H to P0; otherwise, send AAH to P2.
Solution:

```
#include <reg51.h>
sbit mybit=P1^5;
void main(void)
{
mybit=1; //make mybit an input
while (1)
{
if (mybit==1)
P0=0x55;
else
P2=0xAA;
}
}
```

Write an 8051 C program to turn bit P1.5 on and off 50,000 times.
Solution:

```
sbit MYBIT=0x95;
```

```
void main(void)
{
unsigned int z;
for (z=0;z<50000;z++)
{
MYBIT=1;
MYBIT=0;
}
}
```

Write an 8051 C program to get bit P1.0 and send it to P2.7 after inverting it.
Solution:

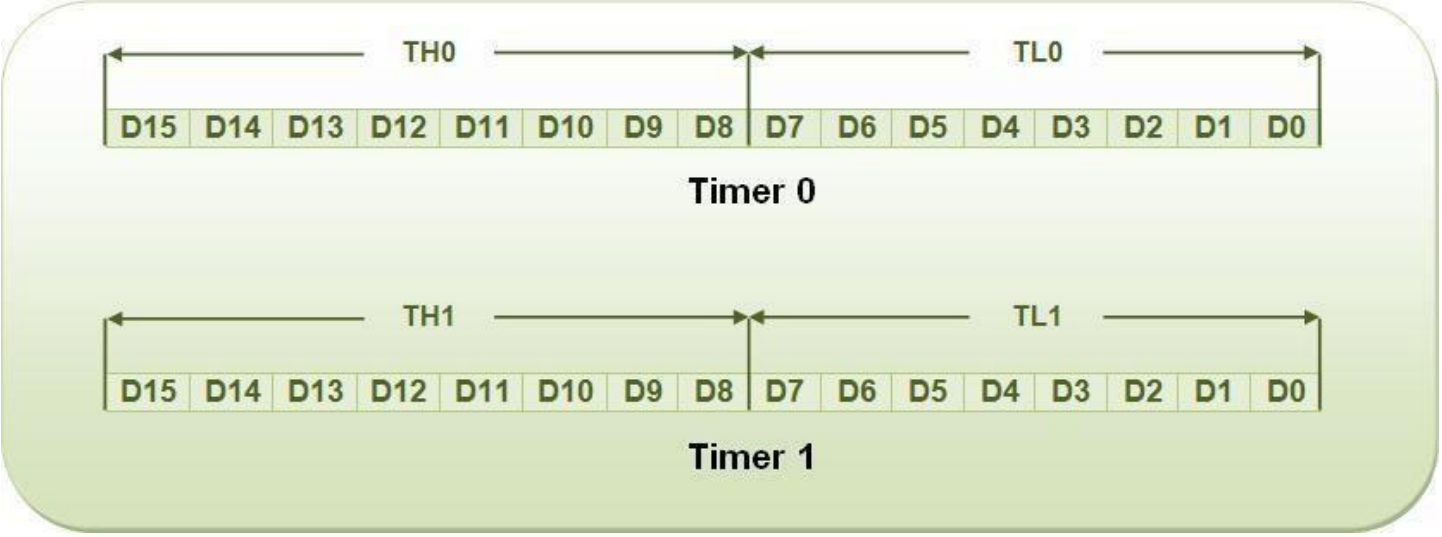
```
#include <reg51.h>
sbit inbit=P1^0;
sbit outbit=P2^7;
bit membit;
void main(void)
{
while (1)
{
membit=inbit; //get a bit from P1.0
outbit=~membit;   //invert it and send it to P2.7
}}
```

3.2 TIMERS AND COUNTERS

The 8051 has two timers/counters, Timer 0 and Timer 1. They can be used as either timers to generate a time delay or as counters to count events happening outside the microcontroller. Both Timer 0 and Timer 1 are 16 bits wide. They are accessed as two separate registers of low byte and high byte.

Timer 0 (T0) Register:
Low byte register of T0 is called TL0 and high byte is called as TH0. These registers can be accessed like any other register. For example, MOV TL0, #3EH, will transfer data 3EH to TL0, and MOV A, TH0 will transfer content of TH0 to accumulator.

Timer 1 (T1) Register:
Low byte register of T1 is called TL1 and high byte is called as TH1. These registers are accessible in the same way as registers of Timer 0.



The following are timer related SFRs in 8051.

SFR Name	Description	SFR Address
TH0	*Timer0 High Byte*	*8Ch*
TL0	*Timer0 Low Byte*	*8Ah*
TH1	*Timer1 High Byte*	*8Dh*
TL1	*Timer1 Low Byte*	*8Bh*
TCON	*Timer Control*	*88h*
TMOD	*Timer Mode*	*89h*

Timers/Counters are used generally for:

a. Time reference
b. Creating delay
c. Wave form properties measurement
d. Periodic interrupt generation
e. Waveform generation

<u>TMOD (Timer Mode) Register:</u>

It is used to various timer operation modes of T0 and T1. It is an 8 bit SFR in which lower 4 bits are used for T0 and upper 4 bits are used for T1.

TMOD : Timer/Counter Mode Control Register (Not Bit Addressable)

GATE	C/$\overline{T}$	M1	M0	GATE	C/$\overline{T}$	M1	M0
TIMER 1				TIMER 0			

GATE When TRx (in TCON) is set and GATE = 1, TIMER/COUNTERx will run only while INTx pin i (hardware control). When GATE = 0, TIMER/COUNTERx will run only while TRx = 1 (so control).

C/$\overline{T}$ Timer or Counter selector. Cleared for Timer operation (input from internal system clock). S Counter operation (input from Tx input pin).

M1 Mode selector bit (NOTE 1).

M0 Mode selector bit (NOTE 1).

Note 1 :

M1	M0	OPERATING MODE	
0	0	0	13–bit Timer
0	1	1	16–bit Timer/Counter
1	0	2	8–bit Auto–Reload Timer/Counter
1	1	3	(Timer 0) TL0 is an 8–bit Timer/Counter controlled by the standard Timer 0 co bits, TH0 is an 8–bit Timer and is controlled by Timer 1 control bits.
1	1	3	(Timer 1) Timer/Counter 1 stopped.